

Curso de Introducción a MFEM/PyMFEM

o cómo usar el Método de los Elementos Finitos en paralelo

Ernesto Aranda

Universidad de Castilla - La Mancha

ernesto.aranda@uclm.es

Sevilla — 16-18 Marzo - 2026

¿Qué es MFEM?

MFEM (Modular Finite Element Method) es una biblioteca en C++ para resolver problemas mediante elementos finitos de alto rendimiento.

- Desarrollada en el Lawrence Livermore National Laboratory (LLNL)
- Licencia de código abierto con licencia BSD
- Diseñada para ser ligera, escalable y de alto rendimiento
- Paralelismo MPI. Soporta GPU
- Interfaces con múltiples librerías: PETSc, SuperLU, STRUMPACK, SuiteSparse, SUNDIALS, ...
- Soporta todo tipo de mallas, AMR, elementos de orden arbitrario, ...
- Visualización a través de GLVis, Paraview y VisIt

¿Qué es PyMFEM?

PyMFEM proporciona enlaces de Python para la biblioteca MFEM mediante SWIG, permitiendo utilizar sus funcionalidades desde Python.

- Ventajas:**
- **Acceso desde Python:** Utiliza MFEM sin necesidad de escribir C++
 - **Prototipado rápido:** Ideal para desarrollo y pruebas
 - **Interactividad:** Uso en Jupyter notebooks

- Desventajas:**
- **Rendimiento:** Python no es C++

Instalación

MFEM

- La versión `serial` solo necesita herramientas básicas de compilación
- La versión `parallel` requiere, además de MPI, *hypr* y METIS
- Se recomienda instalar GLVis (visualizador creado por el LLNL)
- Instrucciones de instalación en mfem.org

PyMFEM

- No es preciso instalar MFEM
- Recomendable instalar en un entorno virtual de Python
- Instalación con `pip`

Docker

- En mfem.org/tutorial/docker hay instrucciones para instalar un contenedor con MFEM
- Para PyMFEM, descargar el fichero `docker-pymfem.sh` desde <https://github.com/e-aranda/Curso-PyMFEM>
- Una vez descargado, desde la terminal de Docker Desktop, moverse hasta la carpeta que contiene el script y ejecutar:

```
./docker-pymfem.sh download  
./docker-pymfem.sh run
```

¿Qué es la Paralelización?

La paralelización es la técnica de dividir un problema computacional en subproblemas que pueden resolverse simultáneamente utilizando múltiples procesadores o núcleos.

Dos tipos de paralelización

Paralelismo de Tareas Diferentes procesos ejecutan *tareas distintas*

Paralelismo de Datos Múltiples procesos ejecutan la *misma tarea* sobre diferentes porciones de datos.

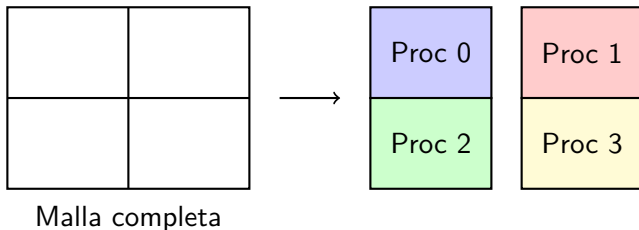
Dos paradigmas en paralelización

Memoria compartida Todos los procesos/hilos acceden al *mismo espacio de memoria* (OpenMP, pthreads)

Memoria distribuida Cada proceso tiene su *propio espacio de memoria* y se comunican por mensajes. (MPI, Message Passing Interface)

Paralelización en MFEM

MFEM utiliza **descomposición de dominio** para distribuir la malla entre diferentes procesos.



Cada proceso trabaja en su subdominio y se comunica con los vecinos para intercambiar información en las fronteras.

La clase `Mesh` es el objeto que gestiona las mallas en MFEM.

- **Lectura desde archivo:**

```
mesh = mfem.Mesh("archivo")
```

Formatos soportados: .mesh (propio), GMSH, VTK, Netgen, etc.

- **Constructores (mallas estructuradas):**

```
mesh = mfem.Mesh(n, L) # malla 1D  
mesh = mfem.Mesh.MakeCartesian2D(nx, ny, tipo,  
    bool, a, b)
```

donde *tipo* se refiere al tipo de elemento: SQUARE, TRIANGLE, etc.
del submódulo `mfem.Geometry`.

Objeto Mesh

Método/Atributo	Descripción
<code>Dimension()</code>	Dimensión de los elementos
<code>GetNE()</code>	Número total de elementos
<code>GetNV()</code>	Número total de vértices
<code>GetNBE()</code>	Número de elementos de frontera
<code>attributes</code>	Etiquetas elementos internos
<code>bdr_attributes</code>	Etiquetas de la frontera
<code>UniformRefinement()</code>	Refinamiento uniforme

Mallas paralelas

`ParMesh(Comunicador, malla serial)`

Clases

Una **clase** es el código que permite generar un objeto.

Componentes

Atributos : características asociadas

Métodos : funciones que trabajan sobre el objeto

```
class Complejo:
    # Constructor: se ejecuta el crear el objeto
    def __init__(self, a, b):
        self.real = a      # Atributo
        self.imag = b      # Atributo

    def conjugate(self): # Método
        return Complejo(self.real, - self.imag)
```

Herencia

La **herencia** permite crear una clase nueva basada en una clase existente, heredando sus atributos y métodos.

```
class ComplejoPolar(Complejo):
    def __init__(self, r, theta):
        from math import sin, cos
        super().__init__(r*cos(theta), r*sin(theta))
        self.r = r
        self.t = theta

    def rotar(self, ang):
        return ComplejoPolar(self.r, self.t + ang)
```

Coeficientes en PyMFEM

En MFEM, un **Coefficient** representa una función matemática susceptible de ser usada en la resolución de un problema. Disponemos de diversos tipos:

Coeficientes Predefinidos

- ConstantCoefficient**(*double*)
- VectorConstantCoefficient**(**Vector**)
- VectorArrayCoefficient**(*dim*)
- PWConstCoefficient**(**Vector**)
- PWCoefficient**(**intArray**, **CoefficientArray**)
- RestrictedCoefficient**(**Coefficient**, **intArray**)
- GridFunctionCoefficient**(**GridFunction**)

Coeficientes calculados

Operaciones con coeficientes

```
SumCoefficient(Coefficient, Coefficient, double, double)
InnerProductCoefficient(VectorCoeff, VectorCoeff)
ProductCoefficient(Coefficient, Coefficient)
PowerCoefficient(Coefficient, double)
```

Definidos mediante funciones analíticas

PyMFEM implementa un nuevo objeto `PyCoefficient` que facilita el uso de coeficientes. Este nuevo objeto requiere implementar el método `EvalValue`.

Calculados a partir de `GridFunction`

Para calcular un coeficiente a partir de una `GridFunction` se implementa un nuevo objeto a partir de `PyCoefficientBase` que requiere implementar el método `Eval`.

Ejemplo I

$$f(x, y) = xy$$

```
class MiFunc(mfem.PyCoefficient):  
    def EvalValue(self, x):  
        return x[0]*x[1]  
coeff = MiFunc()
```

$$F(x, y) = (y, -2x)$$

```
class MiVec(mfem.VectorPyCoefficient):  
    def __init__(self, dim):  
        mfem.VectorPyCoefficient.__init__(self, dim)  
  
    def EvalValue(self, x):  
        u = [ x[1], -2*x[0] ]  
        return u  
veccoeff = MiVec(2)
```

Ejemplo II

$$u^2 - |\nabla u|^2$$

```
class MiCoef(mfem.PyCoefficientBase):
    def __init__(self, u_):
        mfem.PyCoefficient.__init__(self)
        self.u = u_
        self.grad = mfem.Vector()
    def Eval(self, T, ip):
        a = self.u.GetValue(T, ip)
        self.u.GetGradient(T, self.grad)
        b = self.grad.Norml2()
        return a**2 - b**2

coeff = MiCoef(u)
```

Funciones del espacio de elementos finitos: GridFunction

Una `GridFunction` es un vector (hereda de la clase `Vector`) que contiene los coeficientes de la expansión en elementos finitos.

```
# Espacio de elementos finitos
fec = mfem.H1_FECollection(order=1, dim=2)
fes = mfem.FiniteElementSpace(mesh, fec)

# Definición
u = mfem.GridFunction(fes)
v = mfem.GridFunction(fes, u) # shallow copy
w = mfem.GridFunction(u) # deep copy

# Inicialización de valores mediante función
const_coeff = mfem.ConstantCoefficient(0.0)
u.ProjectCoefficient(const_coeff)
u.ProjectBdrCoefficient(const_coeff)
```


Objeto GridFunction

Método/Atributo	Descripción
<code>Size()</code>	Tamaño del vector
<code>GetNodalValues(Vector, comp)</code>	Valores en los nodos
<code>GetValue(---)</code>	Valor en un punto
<code>MakeRef(FE, Vector, index)</code>	Asocia vector con función
<code>ProjectGridFunction(GridFunction)</code>	Proyecta el argumento
<code>ComputeL2Error(Coefficient)</code>	Error en norma 2
<code>ComputeMaxError(Coefficient)</code>	Error en norma ∞

Formulación variacional

El problema básico que resuelve MFEM se formula como

$$a(u, v) = f(v)$$

con a una forma bilineal y f una forma lineal

```
a = mfem.BilinearForm(fespace)
b = mfem.LinearForm(fespace)
```

Añadimos términos a la forma con los métodos

```
AddDomainIntegrator(integr, [atrb])
AddBoundaryIntegrator(integr, [atrb])
AddInteriorFaceIntegrator(integr)
AddBdrFaceIntegrator(integr, [atrb])
```

Integradores comunes

Formas bilineales

- `DiffusionIntegrator`(α): $\int_{\Omega} \alpha \nabla u \cdot \nabla v$
- `MassIntegrator`(ρ): $\int_{\Omega} \rho uv$
- `ConvectionIntegrator`($\mathbf{b}$): $\int_{\Omega} (\mathbf{b} \cdot \nabla u) v$

Visitar <https://mfem.org/bilininteg/>

Formas lineales

- `DomainLFIntegrator`(f): $\int_{\Omega} f v$
- `DomainLFGradIntegrator`($\mathbf{f}$): $\int_{\Omega} \mathbf{f} \cdot \nabla u$
- `BoundaryLFIntegrator`(f): $\int_{\Gamma} f v$

Visitar <https://mfem.org/lininteg/>

Formulaciones Mixtas

Para formulaciones variacionales mixtas usamos:

```
MixedBilinearForm(esp1, esp2)
```

En tal caso, es preciso componer la solución y la matriz del sistema por bloques:

```
block_offsets = intArray([0, dimU, dimP])  
block_offsets.PartialSum()  
x = BlockVector(block_offsets)  
op = BlockOperator(block_offsets)
```

MFEM Solvers

Un Solver en MFEM se asocia a un operador mediante `SetOperator(A)` y se ejecuta con `Mult(b, x)`.

Solver	Tipo de Problema
CGSolver	Simétricos Definidos Positivos (SPD)
GMRESSolver	No simétricos / Indefinidos
BiCGSTABSolver	No simétricos (más ligero que GMRES)
MINRESSolver	Simétricos Indefinidos

Parámetros usuales

- `SetRelTol(double)`: Tolerancia relativa.
- `SetMaxIter(int)`: Máximo de iteraciones.
- `SetPrintLevel(int)`: Nivel de verbosidad (0-3).
- `SetPreconditioner(precond)`: Precondicionador.

Ejemplo

```
# Definición del solver
cg = mfem.CGSolver(comunicador)
cg.SetRelTol(1e-8)
cg.SetMaxIter(200)

# Precondicionador
amg = mfem.HypreBoomerAMG(A)

# Asociar y resolver
cg.SetPreconditioner(amg)
cg.SetOperator(A)
cg.Mult(B, X)
```